

Cross-Modal Emotion-Aligned Latent Space for Video/Text-Conditioned Music Generation

Feasibility Assessment and Implementation Plan

EmoMV Project

July 2, 2026

Contents

Executive Summary	1
1 Current Assets	2
1.1 Video encoder (Stage A, done)	2
1.2 Text encoder (Stage A, done)	3
2 Proposed System Overview	3
3 Stage B: Cross-Modal Alignment	3
3.1 Which embedding to project	3
3.2 Adapter architecture	4
3.3 The core feasibility question: what counts as a “pair”?	4
3.4 Loss function	5
3.5 Training recipe / best practices	5
3.6 Evaluation protocol	6
4 Stage C: Conditioning a Music Generation Model	6
4.1 A resource you already have	6
4.2 How MusicGen conditioning works	7
4.3 Recommended incremental approach	7
4.4 Compute reality check	7
5 Feasibility Summary	8
6 Recommended Roadmap	8
Open Assumptions to Confirm	9

Executive Summary

You currently have two independently trained, frozen-backbone **5-class mood classifiers** — a video model (VideoMAE + CLIP → fusion head, 80.1% test accuracy) and a text model (fine-tuned DistilRoBERTa, macro-F1 ≈ 0.80). The proposed next phase is to bolt a small *projection adapter*

onto each frozen encoder and train the two adapters jointly with a contrastive (InfoNCE/CLIP-style) loss, producing a shared, modality-agnostic embedding space. That space would later condition a fine-tuned MusicGen model to generate music from video or text input.

Verdict: the overall project is feasible in phases, and the next phase (cross-modal alignment) is straightforward with the resources you already have. The one design decision that materially changes the approach — and the central finding of this document — is that **you do not have literal (video, caption) pairs to train a standard CLIP-style contrastive loss on.** What you have instead is two datasets that each carry the *same 5-class categorical label*. Section 3.3 explains why this is still workable (a well-established technique, *label-anchored supervised contrastive alignment*), what it gives you, and what it does *not* give you. Section 4 identifies a resource you already have and may not have noticed: because your video encoder was trained only on EmoMV’s MATCH clips (video emotion = music emotion, by the dataset’s own construction), **every one of your 3,178 training clips already has a mood-consistent music track attached** — which is exactly the paired data Stage C needs.

Stage	Status	Feasibility on your current hardware
A. Unimodal classifiers (video, text)	Done	—
B. Cross-modal projection + contrastive alignment	Proposed next	Feasible
C. MusicGen conditioning / fine-tuning	Future	Feasible, higher risk (compute risk on Apple Silicon)

1 Current Assets

1.1 Video encoder (Stage A, done)

Built and evaluated in the EmoMV preprocessing/training pipeline (project/): frozen VideoMAE-base (768-d, mean-pooled) + frozen CLIP ViT-B/32 (512-d, mean-pooled over 16 frames) → concatenated 1280-d vector → trained fusion head (Linear(1280,512) → GELU → Dropout → Linear(512,5)).

	Accuracy	Macro-F1	Weighted-F1
SlowFast baseline (reference)	50.6%	0.479	0.503
VideoMAE+CLIP fusion (current video model)	80.1%	0.794	0.803

Important terminology note. The VideoMAE and CLIP *backbones themselves are frozen* — only the 512-d fusion head was trained. What you have is not a fine-tuned encoder in the strict sense (no gradients ever reached VideoMAE/CLIP’s weights); it is a frozen general-purpose representation plus a small trained classification head. This distinction does not block Stage B — in fact it is the *recommended* pattern for exactly this kind of adapter-based alignment (Section 3.1) — but it is worth being precise about when writing this up, since “fine-tuned video encoder” and “frozen encoder with a trained head” have different implications for how much the representation has actually specialized to your 5 mood classes.

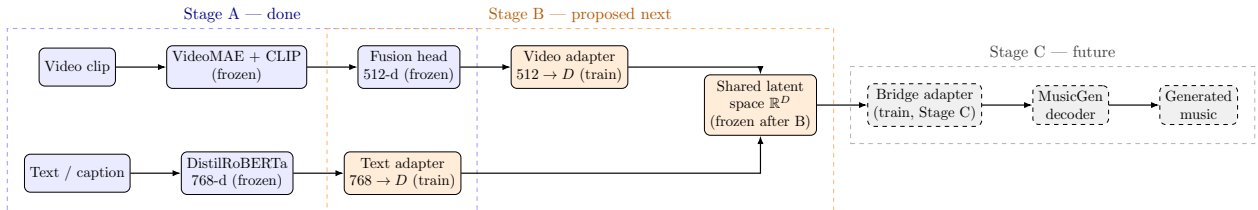
1.2 Text encoder (Stage A, done)

Located at `project/text_mood_clf_encoder/`: `distilroberta-base` (`RobertaForSequenceClassification`, 6 transformer layers, hidden size 768, 12 attention heads, vocab 50,265) **fully fine-tuned** end-to-end for 5-class mood classification. Test macro-F1 ≈ 0.80 . Unlike the video model, this is a genuine fine-tuned encoder — the RoBERTa backbone’s own weights were updated during training.

id	video label	text label
0	exciting	excited
1	fear	fear
2	tense	tense
3	sad	sad
4	relax	relax

The label ordering is identical between the two encoders (only ”exciting” vs. ”excited” differs lexically) — a small but genuinely useful coincidence: any code that treats labels as integer class IDs 0–4 does not need a remapping table between the two encoders.

2 Proposed System Overview



Blue = already frozen/trained. Orange = the next thing to train (small, cheap). Gray/dashed = future work, contingent on Stage B succeeding.

3 Stage B: Cross-Modal Alignment

3.1 Which embedding to project

Do not project the raw 1280-d concatenated VideoMAE+CLIP vector, and do not project RoBERTa’s raw token embeddings. Project the *penultimate*, *task-supervised* representation from each existing classifier:

- **Video:** the 512-d activation after `Linear(1280,512) → GELU → Dropout`, i.e. everything in `FusionClassifier` except the final `Linear(512,5)`.
- **Text:** the 768-d pooled/CLS representation RoBERTa’s classification head consumes, i.e. everything in `RobertaClassificationHead` except the final `out_proj`.

Both are already the result of *supervised* training toward the same 5-class taxonomy, so they start Stage B already partially aligned in a semantic (if not geometric) sense — the projection adapters have to learn a much easier function (a geometric alignment of two already-discriminative spaces) than if they started from generic, task-agnostic backbone features.

3.2 Adapter architecture

Two small MLPs, symmetric in design, asymmetric in input width:

$$\text{video_adapter} : \mathbb{R}^{512} \rightarrow \mathbb{R}^D, \quad \text{text_adapter} : \mathbb{R}^{768} \rightarrow \mathbb{R}^D$$

each implemented as `Linear(in, 2D) → LayerNorm → GELU → Dropout(0.1--0.2) → Linear(2D, D)`, followed by L2 normalization ($\hat{z} = z/\|z\|_2$) so all downstream similarities are pure cosine similarities — the same design CLIP itself uses.

Choosing D . With only 3,178 video clips and a text-side dataset of unknown but likely comparable order of magnitude, a smaller shared space regularizes better.

	$D = 256$ (recommended start)	$D = 512$
Overfitting risk	Lower	Higher on this data volume
Capacity for nuance beyond 5 classes	Lower	Higher
MusicGen bridge (Stage C) input	Smaller, simpler	Larger, more capacity

Start at $D = 256$; only move to 512 if retrieval metrics (Section 3.6) plateau below what a linear probe on the raw 512-/768-d features already achieves — that would indicate the adapters are under-capacity, not that the data is exhausted.

3.3 The core feasibility question: what counts as a “pair”?

Standard CLIP training relies on naturally occurring (image, caption) pairs — millions of them — where each caption *describes that specific image*. **You do not have this.** You have:

- ~3,178 videos, each with one of 5 categorical mood labels.
- An unknown-but-likely-larger text corpus, each sentence with one of the same 5 categorical mood labels.
- No instance-level correspondence between any specific video and any specific sentence.

This rules out literal CLIP-style pairing, but it does *not* rule out contrastive alignment. The applicable technique is **label-anchored supervised contrastive learning** (a cross-modal generalization of Khosla et al., *Supervised Contrastive Learning*, 2020): within a training batch, treat *any* text sample sharing a video sample’s label as a positive, and everything else as a negative. This is a well-established, correct way to train a contrastive objective when you have shared categorical supervision instead of instance-level pairs — it is, in effect, what you would get if you naively used class names as captions (a technique CLIP itself uses at *inference* time for zero-shot classification), generalized to use your own, richer, label-consistent example sentences instead of a single class-name string per class.

What this buys you: a shared space where “a video labeled *relax*” and “a sentence labeled *relax*” land near each other, and the five mood clusters are well separated across both modalities.

What this does *not* buy you: fine-grained semantic alignment within a class. Two very different relaxing scenes (a rainy evening with tea vs. a quiet mountain hike) will be pulled to nearly

the same point in the shared space, because the only training signal they share is the class label *relax*. If your downstream goal is coarse mood-conditioned generation ("make it feel relaxing"), this is entirely sufficient. If you eventually want the shared space to distinguish *shades* of relaxation, you will need richer supervision than 5 discrete labels (e.g. LLM-generated per-clip captions used as weak paired text, or human-written descriptions) — worth flagging now as a scoped-out stretch goal, not a blocker for Stage B.

3.4 Loss function

For a batch of N video embeddings $\{v_i\}$ and N text embeddings $\{t_j\}$ (both L2-normalized), with a learnable temperature τ (parameterized as CLIP does, via `logit_scale` = $\log(1/\tau)$, clamped to $[\log 1, \log 100]$):

$$s_{ij} = \frac{v_i \cdot t_j}{\tau}$$

Simple (single-positive) InfoNCE, if pairing one video to one randomly-sampled same-class text per step:

$$\mathcal{L}_{v2t} = -\frac{1}{N} \sum_i \log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})}, \quad \mathcal{L}_{t2v} = -\frac{1}{N} \sum_i \log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ji})}$$

Recommended: supervised (multi-positive) generalization, using every same-class text in the batch as a positive for a given video anchor i (and symmetrically for text anchors), where $P(i)$ is the set of in-batch opposite-modality samples sharing i 's label:

$$\mathcal{L}_i = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(s_{ip})}{\sum_a \exp(s_{ia})}$$

This uses more of the available signal per batch and is empirically more stable with a small number of classes (Khosla et al., 2020).

Auxiliary classification loss. Add a small linear head on top of each adapter's pre-normalization output and keep the original 5-class cross-entropy loss alive during Stage B training:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contrastive}} + \lambda \left(\mathcal{L}_{\text{CE}}^{\text{video}} + \mathcal{L}_{\text{CE}}^{\text{text}} \right), \quad \lambda \approx 0.1\text{--}0.3$$

This is not optional decoration — with only 5 anchor classes, a pure contrastive objective is prone to representation collapse (all points of one modality contracting toward a single vector per class with no useful spread). The auxiliary CE term anchors the geometry and gives you an easy, already-familiar validation metric (in-space classification accuracy) alongside the retrieval metrics below.

3.5 Training recipe / best practices

- Keep VideoMAE, CLIP, and RoBERTa **fully frozen** throughout Stage B. Only the two adapters (and their small auxiliary classification heads) are trainable. This preserves the Stage A investment and keeps training cheap.
- Precompute and cache the 512-d video / 768-d text penultimate embeddings once (analogous to the existing `features/videomae`, `features/clip` caching pattern in the current pipeline) — Stage B training then never touches raw video frames or a tokenizer again, so each epoch is a few small matrix multiplies. Fully feasible, fast, on Apple Silicon (MPS) or even CPU.

- Batch construction: sample class-balanced batches (e.g. k videos and k texts per class, 5 classes $\rightarrow$ batch size $5k$) rather than uniform random sampling, so every batch has a reasonable positive pool for every anchor.
- Use AdamW, small learning rate (adapters are small — 1e-3 to 3e-4 is a reasonable starting range), cosine or step LR decay, early stopping on the validation retrieval metric (not the loss).
- Log τ (temperature) over training — if it saturates against its clamp bounds, that is a diagnostic signal (either the embeddings are collapsing, or the space is too easy to separate, which is not implausible with only 5 classes — treat as informative, not necessarily broken).

3.6 Evaluation protocol

Classification accuracy alone is not the right metric here — it tells you the adapters preserved discriminability, not that the two modalities are actually *aligned* in the shared space. Report all of:

1. **Cross-modal retrieval:** video $\rightarrow$ text and text $\rightarrow$ video Recall@1 / Recall@5, computed as "does the nearest neighbor(s) in the other modality share the true label."
2. **Linear probe accuracy** on frozen shared embeddings (sanity check against Stage A numbers — should not regress much).
3. **Modality gap diagnostic:** mean cosine similarity between each class's video-centroid and text-centroid. CLIP-family models are well documented to retain a persistent "modality gap" (the two modalities occupy separate sub-manifolds even after contrastive training) — measuring this explicitly, rather than assuming the loss fully closes it, is good practice and directly relevant to whether Stage C's bridge adapter needs to compensate for a gap.
4. **Qualitative:** t-SNE/UMAP projection of the shared space, colored by (modality, label) — look for same-label points from both modalities clustering together, not two same-colored-but-separate modality islands.

4 Stage C: Conditioning a Music Generation Model

4.1 A resource you already have

Recall the EmoMV preprocessing decision (documented in the project's own EDA and manifest pipeline): training used only **MATCH** clips, where *video emotion equals music emotion by the dataset's own construction* — that is what "matched" means in EmoMV. Every one of the 3,178 clips behind your video encoder therefore has an audio track whose mood is, by dataset design, consistent with the video's mood label. **This is a ready-made (mood label, music audio) paired dataset for Stage C fine-tuning**, extractable directly from the same .mp4 files already on disk (audio track $\rightarrow$ waveform via `ffmpeg`) — no new data collection needed to get started.

Two caveats worth resolving before relying on this data for generation training:

- These are *music-video soundtracks*, which may include vocals, sound effects, or dialogue — not necessarily clean instrumental beds. If the target use case is instrumental background-music generation, budget time to filter or vocal-separate (e.g. a lightweight vocal-activity classifier, or an off-the-shelf source-separation model) before using this audio as a fine-tuning target.

- Audio quality/bitrate in EmoMV is modest (EDA measured a median bitrate around 670 kbps, h264/aac container) — adequate for mood-conditioning fine-tuning, but re-encode/resample to whatever sample rate MusicGen’s EnCodec tokenizer expects (32 kHz) rather than assuming the source rate matches.

4.2 How MusicGen conditioning works

MusicGen (Meta, via the `audiocraft` library, also available through `transformers`’ `MusicgenForConditionalGeneration`) is an autoregressive transformer over EnCodec discrete audio tokens. Text conditioning is injected via a frozen T5 text encoder whose hidden-state sequence is cross-attended by the decoder — MusicGen does not consume a single vector, it consumes a *sequence* of conditioning embeddings.

This is the key architectural fact for Stage C: your shared space produces a single D -dim vector per input, but MusicGen’s cross-attention expects a $(T, 768)$ -shaped sequence (T5’s hidden size). The bridge adapter’s job is exactly this reshaping/re-projection, not just a linear resize.

4.3 Recommended incremental approach

Do not start by fine-tuning MusicGen’s weights. Validate the idea cheaply first:

1. **Milestone 1 — embedding translation only (cheap, fast, do this first).** Freeze *everything*, including MusicGen. Train only a small bridge network $\mathbb{R}^D \rightarrow \mathbb{R}^{T \times 768}$ (e.g. a linear layer broadcasting/tiling the vector across T positions, or a tiny transformer decoder with learned positional queries) that maps a shared-space embedding into MusicGen’s existing T5-conditioning slot, *as if it were a text embedding*. Train this bridge with a reconstruction/likelihood objective against the paired (mood label, music audio) data from Section 4’s first paragraph, using MusicGen’s own frozen decoder to score generated tokens. This tells you, cheaply, whether the shared space carries usable conditioning signal at all before you invest in touching MusicGen’s weights.
2. **Milestone 2 — lightweight adapter fine-tuning (only if Milestone 1 underperforms).** LoRA or adapter layers on MusicGen’s cross-attention blocks specifically (not full fine-tuning), still keeping the LM backbone frozen. `audiocraft` and `transformers` both support plugging in custom conditioners; this is the intended extension point, not a hack.
3. **Milestone 3 — full/partial fine-tuning (stretch goal).** Only justified if Milestone 2 plateaus and more paired data becomes available (e.g. supplementing EmoMV audio with a public mood-tagged music corpus such as MTG-Jamendo).

4.4 Compute reality check

Stages A and B are fully tractable on the current Apple Silicon machine (frozen backbones, small trainable adapters, cached embeddings). Stage C is a different story:

- `audiocraft`’s training code has historically assumed CUDA-specific kernels (mixed precision, custom attention paths) with inconsistent MPS support. The `transformers` port of MusicGen (`MusicgenForConditionalGeneration`) has better MPS compatibility for *inference* and simple fine-tuning loops, and is the safer entry point for Milestone 1.

- Even the smallest MusicGen checkpoint plus an EnCodec tokenizer plus gradient state for fine-tuning is a meaningfully larger memory/compute footprint than anything in Stages A/B. Milestone 1 (bridge-only, MusicGen fully frozen, so no optimizer state on its $\sim 300\text{M}+$ parameters) is the only Stage C sub-task realistically safe to attempt purely locally.
- **Recommendation:** budget for a rented cloud GPU session (a single mid-tier GPU for a few hours to a day) once you reach Milestone 2, rather than assuming local Apple Silicon compute will carry the whole project through to a fine-tuned generator.

5 Feasibility Summary

Stage	Verdict	Key resource you have	Key open risk
B: Adapters + contrastive alignment	Feasible	Two already-trained 5-class classifiers on the same label taxonomy; cheap to cache embeddings and train small adapters locally	Only 5 discrete anchor classes $\rightarrow$ risk of representation collapse; mitigated with auxiliary CE loss and retrieval-based evaluation
C, Milestone 1: bridge/conditioner only	Feasible/Feasible, higher risk	EmoMV MATCH clips already give mood-consistent (video, music) pairs at no extra collection cost	Soundtrack audio may include vocals/dialogue; needs a filtering or acceptance decision
C, Milestone 2+: adapter/LoRA or full fine-tuning of MusicGen	Resource-limited	Bridge from Milestone 1 as a validated starting point	audiocraft/MPS compute limits; likely needs rented GPU time; 3.2k clips is a small generative fine-tuning set

6 Recommended Roadmap

1. Extract and cache the 512-d video / 768-d text penultimate embeddings for every existing labeled example (video and text, independently) — mirrors the existing feature-caching pattern already used for VideoMAE/CLIP.
2. Implement the two adapters + label-anchored supervised contrastive loss (Section 3.3) with the auxiliary CE term; start with $D = 256$.
3. Train, then evaluate with the full protocol in Section 3.6 — retrieval R@1/R@5, linear probe, modality gap, and a t-SNE plot — before declaring Stage B done. Classification accuracy alone is not sufficient evidence of alignment.
4. **Gate:** only proceed to Stage C once retrieval R@1 clearly exceeds chance (20% for 5 balanced classes) by a wide margin and the modality gap diagnostic shows real cross-modal proximity,

not two separated same-label clusters.

5. Extract audio tracks from the existing `MATCH` clips as Stage C’s initial paired dataset; decide on the vocals/instrumental question up front.
6. Stage C Milestone 1 (bridge-only, everything else frozen) via `transformers`’ MusicGen port, locally.
7. Re-assess compute needs before Milestone 2; budget cloud GPU time if local training proves infeasible (Section 4.4).

Open Assumptions to Confirm

- This plan assumes the text-mood dataset behind `text_mood_clf` has no instance-level correspondence to any EmoMV video. If some correspondence does exist (e.g. the text data was derived from EmoMV metadata/descriptions), the pairing strategy in Section 3.3 should be revisited — real pairs are strictly better than label-anchored pairs where available.
- This plan assumes the downstream goal is *coarse mood-conditioned* generation (5-class granularity), consistent with what both existing classifiers were trained for. If finer-grained semantic conditioning is the actual goal, Stage B’s supervision needs to be richer than 5 discrete labels before it is worth building Stage C on top of it.
- Whether soundtrack vocals are acceptable in Stage C’s generation targets is a product decision, not a technical one, and should be settled before audio extraction begins.